DynamoDB

DynamoDB 是 AWS 提供的一個全受管（fully-managed）、高度可擴展的鍵值服務。聽起來都是行銷術語，但這到底是什麼意思，為什麼它很重要？

- **全受管（Fully-Managed）**：AWS 負責資料庫所有的運維方面，像是硬體配置、設定、修補、擴展，開發者不需要觸碰任何這些配置，可以專注在應用程式開發上。
- **高度可擴展（Highly Scalable）**：DynamoDB 能處理龐大的資料量和流量。它會自動根據你的應用程式需求向上或向下擴展，不會有任何停機或效能下降。
- **鍵值（Key-Value）**：DynamoDB 是 NoSQL 資料庫，不使用傳統的關聯式資料庫模型，而是採用鍵值模型，讓資料的儲存和取得更有彈性。

簡單說，DynamoDB 非常好上手，又能擴展支援各式各樣的應用程式。就系統設計面試而言，它幾乎涵蓋了你對資料庫的所有需求。它甚至現在已經支援交易（transactions）了，這讓過去對 DynamoDB 最大的批評失去了根據。

值得一提的是，DynamoDB 不是開源的，所以我們沒辦法像分析 Kafka 或 Redis 那樣深入剖析它的內部原理。我們的重點會放在如何使用它，以及 AWS 透過官方文件和 DynamoDB 論文所揭露的有限資訊。

> **面試者常問：「面試裡可以用 DynamoDB 嗎？」**
> 答案很簡單：問你的面試官！很多面試官會說可以，並期待你知道怎麼用它。也有些面試官希望避開供應商鎖定，期待你用開源的替代方案。永遠直接問就對了。

資料模型

DynamoDB 的資料組織成表格（table），每個表格包含多個項目（item），代表各別的記錄。這和關聯式資料庫類似，但有幾個為了擴展性和彈性而設計的關鍵差異。

- **Table（表格）**：DynamoDB 的最頂層資料結構，每個表格都有一個必填的主鍵（primary key）來唯一識別其中的項目。表格支援次要索引（secondary index），讓你可以用非主鍵的屬性來查詢資料。
- **Item（項目）**：相當於關聯式資料庫裡的「列（row）」，包含一組屬性（attribute）。每個 item 必須有主鍵，最多可以儲存 400KB 的資料（包含所有屬性，像是屬性名稱就算作大小限制的一部分。）。
- **Attribute（屬性）**：構成 item 內容的鍵值對。類型可以是純量類型（字串、數字、布林值）或集合類型（字串集合、數字集合）。屬性也可以巢狀嵌套，讓單一 item 裡能有複雜的資料結構。

bukmoai.org

設定 DynamoDB 非常直觀：你可以直接在 AWS 控制台建立表格，然後馬上開始插入資料。和傳統的關聯式資料庫不同，DynamoDB 是無綱要（schema-less）的——你不需要在插入資料之前先定義綱要。這表示同一個表格裡的 item 可以有不同的屬性集，任何時候都可以新增屬性，不影響現有的 item。這種無綱要設計提供了極高的彈性，但需要在應用程式層做仔細的資料驗證，因為 DynamoDB 本身不會強制 item 之間的屬性一致性。

```
{
  "PersonID": 101,
  "LastName": "Smith",
  "FirstName": "Fred",
  "Phone": "555-4321"
},
{
  "PersonID": 102,
  "LastName": "Jones",
  "FirstName": "Mary",
  "Address": {
    "Street": "123 Main",
    "City": "Anytown",
    "State": "OH",
    "ZIPCode": 12345
  }
},
{
  "PersonID": 103,
  "LastName": "Stephens",
  "FirstName": "Howard",
  "Address": {
    "Street": "123 Main",
    "City": "London",
    "PostalCode": "ER3 5K8"
  },
  "FavoriteColor": "Blue"
}
```

每個 item 代表一個用戶，有各自的屬性。注意有些用戶有其他人沒有的屬性，例如 `FavoriteColor`，這就展示了 DynamoDB 在屬性管理上的彈性。

buildmoat.org

雖然 DynamoDB 用 JSON 格式傳輸資料，但那只是傳輸格式。DynamoDB 的實際儲存格式是私有的，讓用戶可以專注在資料建模上，而不需要管底層的物理儲存細節。

分區鍵和排序鍵

DynamoDB 的表格由主鍵定義，主鍵可以由一個或兩個屬性組成：

分區鍵（Partition Key）：一個單一屬性，加上排序鍵（如果有的話），唯一識別表格中的每個 item。DynamoDB 用分區鍵的值來決定 item 在資料庫中的物理位置——這個值會被雜湊（hash），用來決定 item 儲存在哪個分區。

排序鍵（Sort Key，選填）：一個額外的屬性，和分區鍵組合在一起形成複合主鍵。排序鍵用來對同一個分區鍵值下的 item 排序，讓你可以在分區內做高效的範圍查詢和排序。

在面試中，介紹 DynamoDB 時一定要指定分區鍵，以及視情況指定排序鍵。這個選擇對查詢效能和資料取得效率至關重要。和任何其他資料庫一樣，要選擇能優化應用程式最常見查詢模式的分區鍵，並讓資料均勻分散在各個分區。如果你需要做範圍查詢或排序，就額外指定排序鍵。

舉個例子：如果你在設計一個群組聊天應用，用 `chat_id` 作為分區鍵、`message_id` 作為排序鍵就很合理。這樣你可以高效地查詢特定聊天群組的所有訊息，並依時間順序排列後顯示給用戶。

注意這裡我們用的是單調遞增的 `message_id`，而不是時間戳記作為排序鍵。雖然時間戳記看起來很直覺，但它不保證唯一性——同一毫秒內可能有多則訊息被建立。單調遞增的 ID 同時提供了時間順序和唯一性。ID 可以用以下技術產生：

- 每個分區的自動遞增計數器
- UUID v7（優於 UUID v1——時間戳記優先的格式讓它作為字串自然可排序，且不暴露機器的 MAC 地址）
- Snowflake ID
- ULID

底層實際發生了什麼？

DynamoDB 結合了雜湊分區和 B-tree 來高效管理資料分散和取得：

分區鍵的雜湊分區：資料的物理位置由分區鍵的雜湊值決定。請求路由器查詢分區 metadata 服務，把雜湊後的鍵對應到正確的儲存節點。這在概念上類似一致性雜湊（consistent hashing），但 DynamoDB 使用的是中心化的分區映射和配置服務（而不是 2007 年原始 Dynamo 論文裡描述的 peer-to-peer hash ring）。分區 metadata 服務也負責處理資料成長時分區的自動拆分和合併。

bumpmost.org

排序鍵的 B-tree：在每個分區內，DynamoDB 用以排序鍵為索引的 B-tree 資料結構來組織 item，讓分區內的範圍查詢和排序取得更有效率。

複合鍵操作：同時使用兩個鍵查詢時，DynamoDB 先用分區鍵的雜湊找到正確的節點，再用排序鍵遍歷 B-tree 找到特定的 item。

這個兩層架構讓 DynamoDB 同時達到水平可擴展性（透過分區）和分區內的高效查詢（透過 B-tree 索引）。這個組合使 DynamoDB 能在處理龐大資料量的同時，仍然對使用分區鍵和排序鍵的查詢提供快速、可預測的效能。

次要索引

如果你需要用不是分區鍵的屬性來查詢資料呢？這就是次要索引（secondary index）發揮功用的地方。DynamoDB 支援兩種類型的次要索引：

全域次要索引（Global Secondary Index，GSI）：使用與表格分區鍵不同的分區鍵（和選填的排序鍵）建立的索引。GSI 讓你能用非表格分區鍵的屬性來查詢 item。由於 GSI 使用不同的分區鍵，資料會儲存在與基底表格完全不同的物理分區上，並分別被複製。

本地次要索引（Local Secondary Index，LSI）：使用與表格主鍵相同的分區鍵，但不同排序鍵的索引。LSI 讓你能在分區內做範圍查詢和排序。由於 LSI 使用和基底表格相同的分區鍵，它們和被索引的 item 儲存在相同的物理分區上。

理解 GSI 和 LSI 在物理儲存上的差異很重要。GSI 維護自己獨立的分區和副本，提供更大的查詢彈性，但需要額外的儲存和處理開銷。LSI 則和基底表格的 item 儲存在一起，讓分區內的查詢更有效率，但靈活度有限。

在實務上，這兩種索引都只需要在 AWS 控制台或 AWS SDK 裡設定就好，DynamoDB 會負責在資料變更時維護和更新這些索引。

什麼時候用 GSI？ 當你需要用非分區鍵的屬性來高效查詢資料時。例如，你的聊天應用有一個訊息表格，主表格的分區鍵是 `chat_id`，排序鍵是 `message_id`，這讓你能輕鬆取得特定聊天群組的所有訊息並按時間排序。但如果你想顯示用戶在所有聊天室裡發送的所有訊息呢？你就需要一個以 `user_id` 為分區鍵、`message_id` 為排序鍵的 GSI。

bullmat.org

Main Table

chat_id	message_id	user_id	num_attachments	content
1	5322	123	0	Hello!
2	7411	124	1	What time is it?
1	8425	125	3	How are you?
1	9327	126	2	lol

Partition Key　Sort Key

GSI created from main table with different primary key and optional sort key

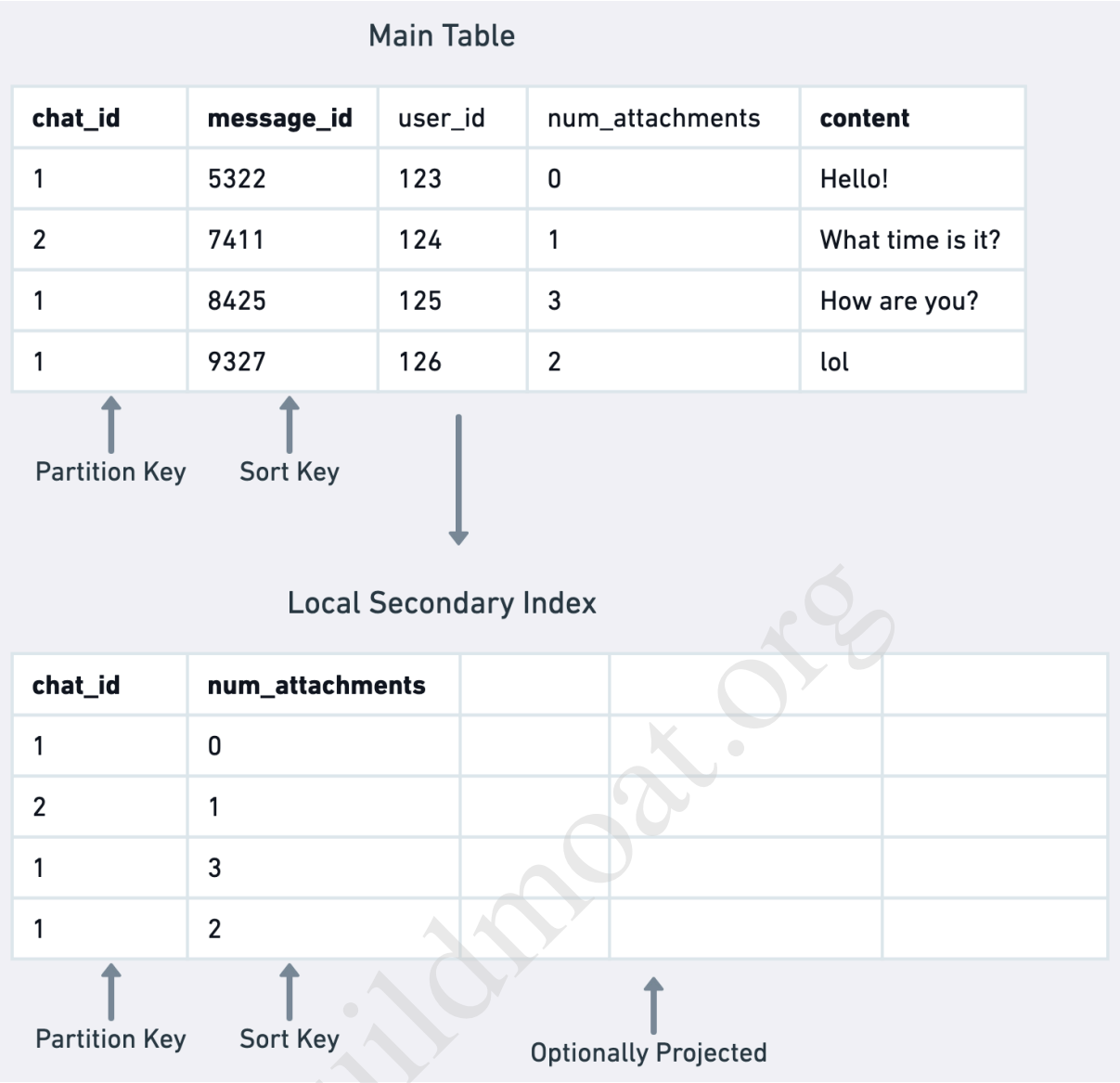
Global Secondary Index

user_id	message_id			
123	5322			
124	7411			
125	8425			
126	9327			

Partition Key　Sort Key　Optionally Projected

buildmoat.org

什麼時候用 LSI？ 當你需要在分區內，用與主要排序鍵不同的屬性做範圍查詢或排序時。回到聊天應用的例子，我們已經可以在聊天群組內按 `message_id` 排序，但如果我們想查詢某個聊天群組裡附件最多的訊息呢？我們可以在 `num_attachments` 屬性上建立一個 LSI，來快速找出附件很多的訊息。一個重要的限制：**LSI 只能在建立表格時定義，之後無法新增或移除**，所以要提前規劃好。



buildmoat.org

	全域次要索引 (GSI)	本地次要索引 (LSI)
定義	使用與主表格不同的分區鍵	使用與主表格相同的分區鍵，但不同的排序鍵
使用時機	需要用非主鍵屬性查詢時	需要在同一個分區鍵下使用額外的排序鍵時
大小限制	無限制	每個分區鍵最多 10GB
吞吐量	與基底表格分開計算讀寫容量	與基底表格共用讀寫容量
一致性	僅支援最終一致性	支援最終一致性（預設）和強一致性讀取
建立時機	可隨時新增或移除	只能在建立表格時定義，無法移除
數量上限	每張表格最多 20 個	每張表格最多 5 個

	全域次要索引 (GSI)	本地次要索引 (LSI)
使用範例	跨所有分區的全域搜尋，例如在用戶資料庫裡用 email 搜尋	分區內的局部搜尋，例如查找某個客戶最近的訂單

底層實際發生了什麼？

次要索引由系統自動維護。GSI 作為獨立的內部表格實作；LSI 則與基底表格共址（co-located）：

GSI：每個 GSI 本質上是一個有自己分區方式的獨立表格。當主表格的 item 被新增、更新或刪除，DynamoDB 會**非同步地**更新 GSI。這允許對所有分區的非主鍵屬性做高效查詢，但意味著 GSI 是**最終一致的**。

LSI：LSI 和主表格的分區共址，使用相同的分區鍵。它們在每個分區內維護一個獨立的 B-tree 結構，以 LSI 的排序鍵為索引。LSI 的更新和主表格的寫入**同步**進行，支援最終一致性（預設）和強一致性讀取。

存取資料

DynamoDB 有兩種主要的資料存取方式：

掃描操作（Scan）：讀取表格或索引中的每一個 item，以分頁的方式返回結果。掃描在你需要讀取整張表格或索引的所有 item 時很有用，但對大型資料集非常低效，因為需要讀取每一個 item，應該盡可能避免使用。

查詢操作（Query）：根據主鍵或次要索引鍵屬性來取得 item。查詢比掃描更有效率，因為只讀取符合指定鍵條件的 item。查詢也可以用來對排序鍵做範圍查詢。

和傳統 SQL 資料庫不同，DynamoDB 的主要介面是透過 AWS SDK 或 AWS 控制台，而不是一個獨立的查詢語言。不過 DynamoDB 確實支援 PartiQL，一個 SQL 相容的查詢語言，讓你可以使用熟悉的 SELECT、INSERT、UPDATE、DELETE 語法。在底層，PartiQL 操作會被轉換成同樣的 DynamoDB 操作，所以它是個方便的語法層，而不是本質上不同的能力。

SQL 查詢：

```
SELECT * FROM users WHERE user_id = 101
```

等效的 DynamoDB 查詢操作：

```
const params = {
  TableName: 'users',
  KeyConditionExpression: 'user_id = :id',
```

fullstack.org

```
  ExpressionAttributeValues: {
    ':id': 101
  }
};

dynamodb.query(params, (err, data) => {
  if (err) console.error(err);
  else console.log(data);
});
```

SQL 全表掃描等效的 DynamoDB 掃描操作：

```
const params = {
  TableName: 'users'
};

dynamodb.scan(params, (err, data) => {
  if (err) console.error(err);
  else console.log(data);
});
```

在使用 DynamoDB 時，你通常要盡一切可能避免昂貴的掃描操作。這正是謹慎的資料建模發揮作用的地方，像是選對分區鍵和排序鍵，確保查詢是高效的。

一個重要的細節：查詢 DynamoDB 時，預設會讀取整個 item。雖然 DynamoDB 支援 `ProjectionExpression` 只回傳特定屬性，但這只能減少網路傳輸量，底層仍然讀取了完整的 item，你也要支付以 item 完整大小計算的讀取容量費用。這和 SQL 的欄位選取是不同的概念。對於大型 item，要適當地正規化資料，避免每次讀取都帶回超過你需要的內容。

CAP 定理

你通常會在面試的非功能性需求階段做一些關於一致性和可用性的早期決策。因此，選擇符合這些需求的資料庫非常重要。

我遇到的大多數面試者，在需要高可用性和擴展性時都選 DynamoDB。這沒有錯，但就像傳統的「SQL vs NoSQL」之爭，這個觀念已經有點過時了。

DynamoDB 支援兩種讀取一致性模型，例如**最終一致性**和**強一致性**。重要的是，這不是表格層級的設定，而是在每個讀取請求上個別指定，透過在 `GetItem`、`Query` 或 `Scan` 呼叫裡設定 `ConsistentRead=true` 來實現。

buildmoat.org

最終一致性（預設）：除非明確要求，否則每次讀取都是最終一致的。這提供了最高的可用性和最低的延遲，但你可能不會立刻看到最近的寫入。DynamoDB 在這個模式下通常表現為 AP 系統，具備 BASE 特性。

強一致性：設定 `ConsistentRead=true` 後，DynamoDB 確保讀取會反映在此次讀取之前所有成功的寫入。這需要兩倍的讀取容量（每 4KB 消耗 1 RCU，而最終一致性只消耗 0.5 RCU），延遲也可能稍高，但保證你看到的是最新資料。

這種每個請求都可以選擇的靈活性，意味著你可以在需要強一致性的場景（例如訂位系統）使用 DynamoDB，同時對讀取頻繁、延遲敏感的路徑保持最終一致性。DynamoDB 也透過 `TransactWriteItems` 和 `TransactGetItems` 支援 ACID 交易，在跨多張表格的最多 100 個 item 上提供可序列化隔離（serializable isolation）。

重要限制：強一致性讀取只支援基底表格和本地次要索引（LSI）。**全域次要索引（GSI）只支援最終一致性讀取**，在設計需要強一致性的存取模式時要牢記這一點。

底層實際發生了什麼？

DynamoDB 的一致性模型透過其分散式架構和複製機制來實現：

最終一致性讀取：讀取可以由分區複製群組中的任意三個副本之一來服務。由於 leader 節點在確認了 quorum（多數）之後，才非同步地把寫入複製給 follower，follower 可能還沒有最新的寫入。消耗較少的讀取容量（每 4KB 0.5 RCU），延遲也較低。

強一致性讀取：讀取請求直接路由到分區的 leader 節點。由於所有寫入都先經過 leader，它永遠有最新的資料。消耗較多的讀取容量（每 4KB 1 RCU），延遲可能稍高，且不支援 GSI。

架構與可擴展性

擴展性

DynamoDB 透過自動分片（auto-sharding）和負載平衡來擴展。當一個分區達到容量上限（大小或吞吐量），DynamoDB 會自動把它拆分，並重新分配資料。雜湊分區確保了節點之間的均勻分配。

AWS 的全球基礎設施進一步強化了這種可擴展性。**全域資料表（Global Tables）** 讓資料可以在多個 Region 之間即時複製，讓全球用戶都能就近做讀寫操作，降低延遲、改善用戶體驗。DynamoDB 也整合了每個 Region 的多個可用區域（Availability Zone），確保資料的持久性和服務的連續性。

在面試中設計全球性應用時，提到用 Global Tables 做跨區域複製通常就夠了。

容錯和可用性

buildmost.org

DynamoDB 透過分散式架構和資料複製機制提供高可用性和容錯能力。服務自動在一個 Region 內的多個可用區域之間複製資料，確保資料在硬體故障或網路中斷時仍然可存取。

DynamoDB 自動在一個 Region 內的**三個可用區域**之間複製資料——這不是用戶可設定的。每個分區維護三個副本（一個 leader 和兩個 follower），完全由 AWS 管理。對於跨 Region 的複製，可以啟用 Global Tables 在其他 AWS Region 新增副本。

在底層，每個分區使用 Multi-Paxos 共識，由一個有三個節點的 leader-based 複製群組管理。Leader 處理所有寫入：產生一筆 WAL 記錄，發送給 peer 節點，等到 quorum（3 個中的 2 個）確認持久化後，寫入就算完成。對於強一致性讀取，DynamoDB 把請求直接路由到 leader；對於最終一致性讀取，三個副本的任何一個都能服務請求，延遲更低，但可能返回稍舊的資料。

安全性

DynamoDB 預設對所有資料靜態加密（at rest），資料在不被存取時也是安全的。DynamoDB 也強制所有 API 呼叫使用 TLS 加密傳輸中的資料，不需要任何額外設定。

DynamoDB 整合了 AWS IAM，提供對資料的細粒度存取控制。你可以建立 IAM 策略，指定誰可以存取資料、以及可以執行哪些操作。你也可以使用 VPC 端點（VPC Endpoint），讓你在 VPC 內部安全地存取 DynamoDB，而不需要把資料暴露到公共網路上。

在面試中，處理敏感用戶資料時，提到你了解 DynamoDB 預設靜態加密、並透過 TLS 強制傳輸加密就夠了。超過這個範圍大概就過頭了。

定價模型

定價聽起來好像跟面試毫不相干，但請繼續看下去，理解定價模型會帶出對架構的明確限制。

DynamoDB 有兩種定價模式：**隨需計費（on-demand）和預置容量（provisioned capacity）**。隨需計費按請求次數收費，適合流量不可預測的工作負載。預置容量則需要用戶指定讀取和寫入容量單位，按小時計費，對於流量可預測的工作負載更划算，但在低流量期間可能有閒置容量的浪費。

計費基於 AWS 所說的**讀取容量單位（Read Capacity Unit，RCU）和寫入容量單位（Write Capacity Unit，WCU）**：

- **1 RCU** = 每秒讀取最多 4KB 的資料（強一致性讀取）；最終一致性讀取消耗 0.5 RCU
- **1 WCU** = 每秒寫入最多 1KB 的資料

buildmoat.org

功能	費用	說明
讀取容量單位 (RCU)	每百萬次讀取約 1.12 美元（每次 4KB）	每秒提供一次強一致性讀取（≤4KB），或兩次最終一致性讀取
寫入容量單位 (WCU)	每百萬次寫入約 5.62 美元（每次 1KB）	每秒提供一次寫入（≤1KB）

費用本身在面試中不是重點，但對數字有基本概念是有用的。每個 DynamoDB 分區最多支援 **3,000 RCU** 和 **1,000 WCU**。這意味著單個分區可以處理每秒 12MB 的讀取（3,000 × 4KB）和每秒 1MB 的寫入（1,000 × 1KB）。DynamoDB 自動處理分片和自動擴展，但這些數字對粗略估算很有幫助。

舉個例子：假設你要在 DynamoDB 儲存 YouTube 的觀看次數。每次寫入（不管多小）至少消耗 1 WCU，因為 DynamoDB 的計費會無條件進位到最近的 1KB。每個分區支援約 1,000 次寫入/秒。如果你預期每秒 1,000 萬次觀看，大約需要 10,000 個分區。以預置容量定價計算（約 0.00065 美元/WCU-小時），那就是 10,000,000 WCU × 0.00065 美元/小時 × 24 小時 ≈ **每天 156,000 美元**。隨需計費會更貴。這種估算能幫你快速判斷你的方案在成本上是否合理。

進階功能

DAX（DynamoDB Accelerator）

DynamoDB 有一個專門設計的記憶體快取，叫做 **DAX（DynamoDB Accelerator）**。這意味著你可能不需要在架構裡引入額外的服務（Redis、Memcached）。

DAX 是專為提升 DynamoDB 效能而設計的快取服務，對讀取密集型工作負載提供微秒級的回應時間。使用 DAX 需要把你的 DynamoDB 客戶端換成 DAX 客戶端 SDK（支援 Java、.NET、Node.js、Python、Go）——API 是相容的，所以改動很小，但並不是完全透明的。

DAX 同時作為讀取式快取（read-through cache）**和**寫入式快取（write-through cache）運作：它快取讀取結果直接提供給應用程式，也把資料同時寫入快取和 DynamoDB。一個重要的細節是：DAX 只對透過 DAX 本身執行的寫入自動讓快取失效。如果你直接更新 DynamoDB（繞過 DAX），那些快取的條目可能持續保留到 TTL 過期或被逐出為止。

DAX 維護兩個快取：一個是 item 快取（給 GetItem/BatchGetItem 結果），另一個是查詢快取（給 Query 和 Scan 結果）。這兩個快取永遠都是啟用的。**重要限制：DAX 不快取強一致性讀取**——當你透過 DAX 請求強一致性讀取時，它會直接把請求傳給 DynamoDB，回傳結果但不快取。

buildmoat.org

Streams（串流）

DynamoDB 也內建支援**變更資料擷取（Change Data Capture，CDC）**，透過 **DynamoDB Streams** 實現。Streams 會捕獲表格中 item 的變更，讓下游應用程式能即時處理。表格中發生的任何變更事件（插入、更新、刪除），都會被記錄為一筆串流記錄供下游消費。

幾個常見的使用場景：

與 Elasticsearch 保持一致：DynamoDB Streams 可以用來讓 Elasticsearch 索引和 DynamoDB 表格保持同步，這對在 DynamoDB 資料上建構搜尋功能很有幫助。

即時分析：你可以在 DynamoDB 表格上啟用 Kinesis Data Streams，然後透過 Kinesis Data Firehose 把變更資料匯入 S3、Redshift 或 OpenSearch 做即時分析。（注意：Firehose 無法直接讀取 DynamoDB Streams，需要 Kinesis Data Streams 或 Lambda 函式作為中介。）

變更通知：你可以用 DynamoDB Streams 觸發 Lambda 函式，回應資料庫的變更——例如發送通知、更新快取，或根據資料變更執行其他操作。

DynamoDB 在面試中的應用

什麼時候用它

在面試中，幾乎任何持久化層的需求你都可以合理地選擇 DynamoDB。它高度可擴展、持久、支援交易，並提供個位數毫秒的延遲（有了 DAX 甚至是微秒級）。DAX 的快取功能和 DynamoDB Streams 的跨系統一致性讓它更加強大。所以如果面試官允許，它很可能是個很好的選擇。

知道它的限制

不過，知道什麼時候**不**用 DynamoDB 同樣重要。以下是幾個你可能選擇其他資料庫的理由（除了只是對其他技術更熟悉之外）：

成本效益：DynamoDB 的定價基於讀寫操作和儲存的資料量，在高流量工作負載下可能很昂貴。如果你需要每秒數十萬次的寫入，成本可能遠超過好處。

複雜的查詢模式：如果你的系統需要複雜的查詢，例如 JOIN 或臨時聚合（ad-hoc aggregation），DynamoDB 可能力不從心。DynamoDB 確實支援跨多張表格的交易（每次交易最多 100 個 item），但缺乏 SQL 資料庫靈活的查詢能力。

資料建模的限制：DynamoDB 要求仔細的資料建模才能有好的效能，它為鍵值和文件結構做了優化。如果你發現自己頻繁地使用 GSI 和 LSI，像 PostgreSQL 這樣的關聯式資料庫可能更合適。

bullmoat.org

供應商鎖定（Vendor Lock-in）：選擇 DynamoDB 意味著綁定在 AWS 上。很多面試官希望你保持雲端供應商中立，所以你可能需要考慮開源的替代方案來避免被綁定。

總結

DynamoDB 功能多元、強大，而且非常好用。在面試中，它是大多數使用案例的穩固選擇，但你要清楚它的限制，並對它的運作方式有扎實的理解——包括如何選擇正確的資料模型、分區鍵、排序鍵、次要索引，以及知道什麼時候要啟用 DAX 和 Streams 等進階功能。

記住：DynamoDB 現在已經支援交易（包括跨多張表格的交易），所以「NoSQL 就是沒有交易」這個舊評論已經過去了。

buildmoat.org